

Putting a Proportional Board on a Synchronised CAN Bus

Firmware architecture, protocol and control law for the TLE board in a mixed 24-actuator system

ESP32-S3 · MCP25625 · TLE92464ED · 2 × Clippard DVP proportional valves · LTC1864 · firmware 1.46 + CAN layer
Every number below is measured on the bench hardware, not estimated.

Summary. A robot arm runs 24 pneumatic actuators from one CAN bus at 150 Hz. Its top eight actuators are being replaced with a proportional-valve manifold on a new board, while the lower sixteen keep their original on/off boards. The two must share a wire, a host and a synchronisation edge. The work reported here adds an old-protocol-compatible layer to the new board's firmware, alongside the richer command set the proportional controller needs, and a broadcast firmware-update path that flashes every board on the bus from one stream. Measured on hardware: 3000 of 3000 sync edges answered over 20 s, reply latency 1.07 ms median and 1.91 ms worst, host cycle jitter 0.001 ms at the 95th percentile, and a 324 kB image broadcast in 21.9 s with no retransmission. Three defects were found and fixed on the way, all of them already present in the firmware and latent because no path had exercised it on a live bus.

1 The problem

The existing system is a solved one: sixteen boards driving 7 mm on/off valves, eight driving larger DT valves, all answering a PC backend that owns a 150 Hz cycle. Replacing the top eight with proportional control does not get to renegotiate any of it. So the

protocol is not a design choice here; it is a constraint, reproduced byte for byte.

What is a choice is where to put everything a proportional board has to say that will not fit in a two-byte frame. It goes to an identifier of its own, `base + 0x400`, which no old board listens to and no part of the old protocol uses.

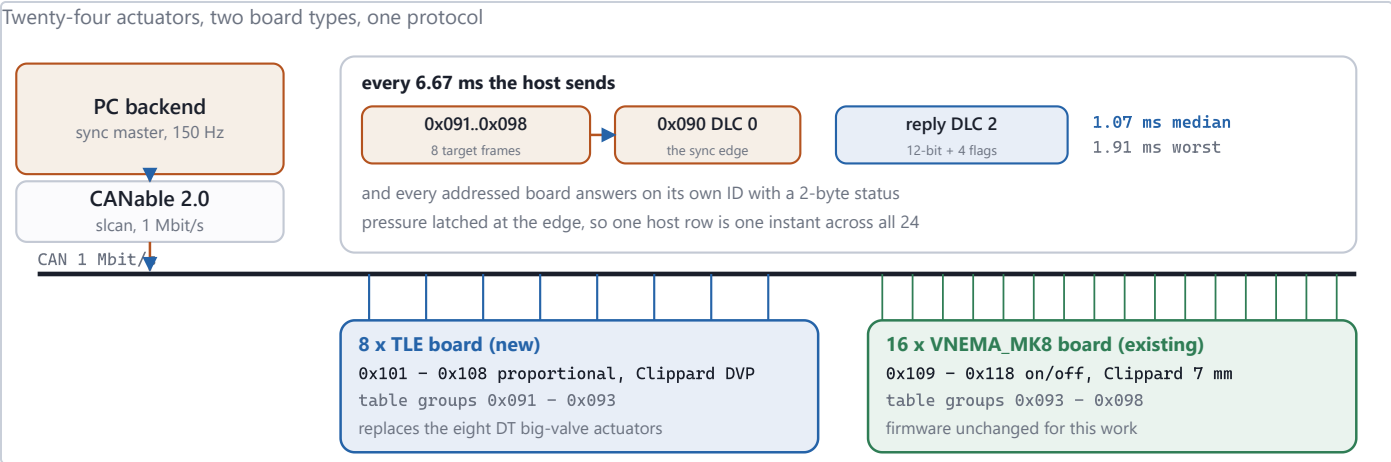


Figure 1. The bus. One wire, 1 Mbit/s, 11-bit identifiers, 120 R at both ends. The host is the only clock. Each cycle it packs 24 targets into 8 frames, then sends a zero-length frame on 0x090 — the sync edge. Every addressed board promotes its staged target and latches the pressure it will report in the same handler, so one row of host data is one instant across the whole arm, even though the replies arrive spread over the following millisecond and a half.

2 What runs where

Everything hangs off one 6 kHz timer. The sampler, the control law and the valve update must hold a fixed phase relationship, and three free-running timers would drift apart; dividing one tick

makes that impossible by construction, and the divisors are checked at compile time.

The cores split along the two SPI buses, which on this board are physically separate. That is what makes the split worth having: a burst of CAN frames cannot delay a valve update, and a slow ADC transaction cannot delay the sync edge.

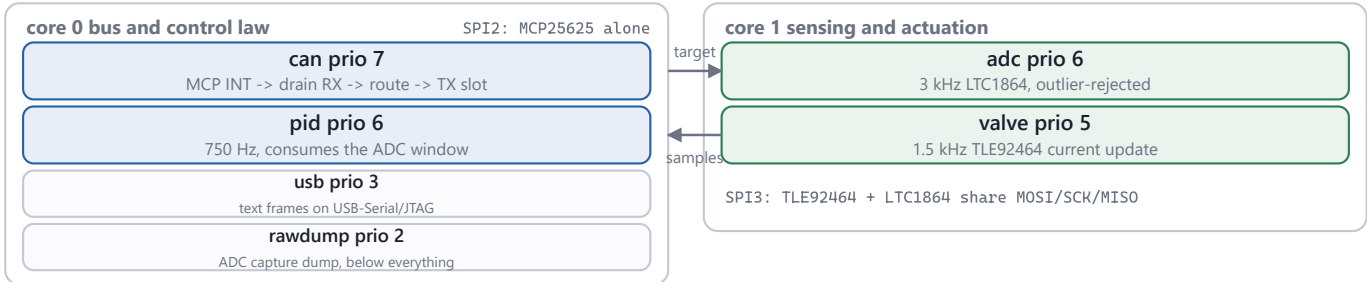
One 6 kHz timer is the only clock; the two cores split the bus from the plant

GPTimer base tick 6000 Hz

/2 -> ADC 3 kHz

/4 -> MAX update 1.5 kHz

/8 -> PID 750 Hz



Why the split is this way

Separate SPI buses, so CAN traffic and valve current updates never queue behind each other.

CAN outranks the control law: the sync edge defines when the whole robot changes target, so servicing it late is worse than running the loop late.

Figure 2. Tasks, cores and rates. A late control step costs one 1.3 ms period of regulation; a late sync edge costs the whole arm its notion of *now*. Hence the priority order. The two lowest tasks exist to be starved — a multi-second ADC dump must never delay anything above it.

3 The cycle

A board does nothing with a target when it arrives. Commands are staged; the sync edge promotes them, and in the same handler the board latches its filtered pressure. That latched value is what the reply carries, deliberately decoupled from when the reply is transmitted, so replies arriving at different times still describe one instant.

Nothing staggers the replies. Every addressed board answers at once and CAN arbitration sorts them out, ordering them by identifier: on the reference 24-board system that spread ran from 2.04 ms at 0x101 to 3.49 ms at 0x118, about 65 μ s per identifier step. The one-board figure below should be added to that spread, not compared with it.

One cycle drawn to scale: 6.67 ms at 150 Hz

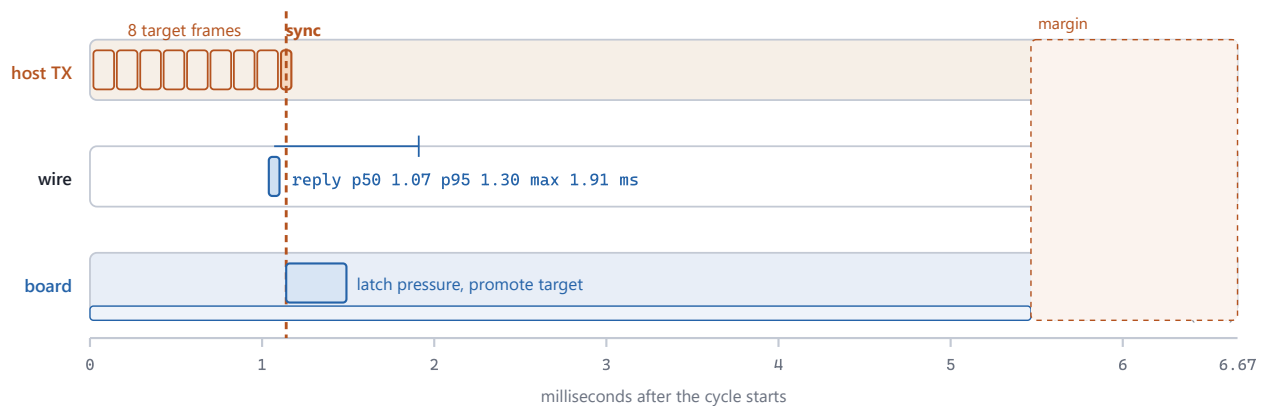


Figure 3. One cycle, to scale. Measured over 1500 consecutive cycles. The host receive window closes at 82 % of the period — the value the original 24-board backend uses — leaving 1.2 ms of margin to keep the next transmit on schedule.

This latency is easy to measure wrongly. The first host reader used a blocking serial read with a 5 ms timeout. Such a read waits out its whole timeout whenever fewer bytes are available than were asked for, so arrivals were batched onto a 5 ms beat and stamped at the end of it. That beat is uncorrelated with the 6.67 ms cycle, so latency smeared uniformly across the period and 41 % of replies looked like they had missed their window. Nothing was being lost — all 1200 arrived. Polling in_waiting instead moved the median from 3.31 ms to 1.07 ms.

4 Protocol and bus load

Pressure on the wire is a 12-bit field, which is what the original 12-bit ADC boards report directly. This board's sensor path is 16-bit, so the compact value is the raw reading shifted right by four: one count is 0.0165 psi, finer than the old boards' resolution and about a third of this controller's default deadband. Absolute pressure still comes from the host's per-actuator calibration, as it does for the old boards — a replacement actuator needs recalibrating anyway, since sensor and plumbing both change.

Identifier map, and the two frames that carry the control loop

0x090	broadcast	DLC 0 is the sync edge. DLC > 0 is OTA image data.
0x091 – 0x098	target table	Three actuators per frame, so 24 targets cost 8 frames.
0x101 – 0x118	actuator base	Direct command in, compact status out.
base + 0x100	host control	OTA start/end, set ID, diagnostics, version.
base + 0x200	unicast data	Repairs one board that fell behind the broadcast.
base + 0x300	board status	Acknowledgements, diagnostics, extended telemetry.
base + 0x400	extended command	TLE only: gains, shaping, current codes. New.

compact command / status DLC 2, little-endian

bits 0..11	bits 12..15
pressure, 12-bit counts	flags

one count = 0.0165 psi on a TLE board (raw >> 4)

target table DLC 8

byte 0	byte 1	2..3	4..5	6..7
0x80 slot	3-bit mask	slot + 0	slot + 1	slot + 2

bit 7 of byte 0 separates a table frame from OTA data, whose first byte is a sequence number 0..127

Figure 4. Identifiers and frames. Everything except the last row is the original protocol, unchanged. `base + 0x400` is new. Over USB, where nothing is shared, the new command set keeps its original identifier so every existing bench tool still works.

Bitrate is a property of the wire, not a per-board choice, and the sixteen existing boards are built for 1 Mbit/s. The load calculation

agrees on the merits: at 500 kbit/s the worst case leaves no room for retransmission, diagnostics or an update running alongside.

BUS LOAD PER 150 HZ CYCLE	1 MBIT/S	500 KBIT/S	VERDICT
24 boards (8 table frames + 1 sync + 24 replies)	37–44 %	73–89 %	1 Mbit/s required
8 TLE boards alone	13–16 %	27–32 %	either works

5 The control law

The proportional loop runs at 750 Hz in fixed point, from a trimmed window of the four 3 kHz samples taken since the last step. Every stage exists because of a measured behaviour of this plant: the gain schedule divides by measured pressure because loop gain rises with it; the soft zone fades P and I in near the

target so a high proportional gain can hold the valve open through a slew without turning the hold into a limit cycle; the derivative damps the approach and fades inside the capture band so sensor noise does not become valve motion; the feed-forward seed exists because this bladder leaks 2–3 psi/s and the integral would otherwise rediscover that on every step.

The proportional loop: 750 Hz, fixed point, and what each stage is for

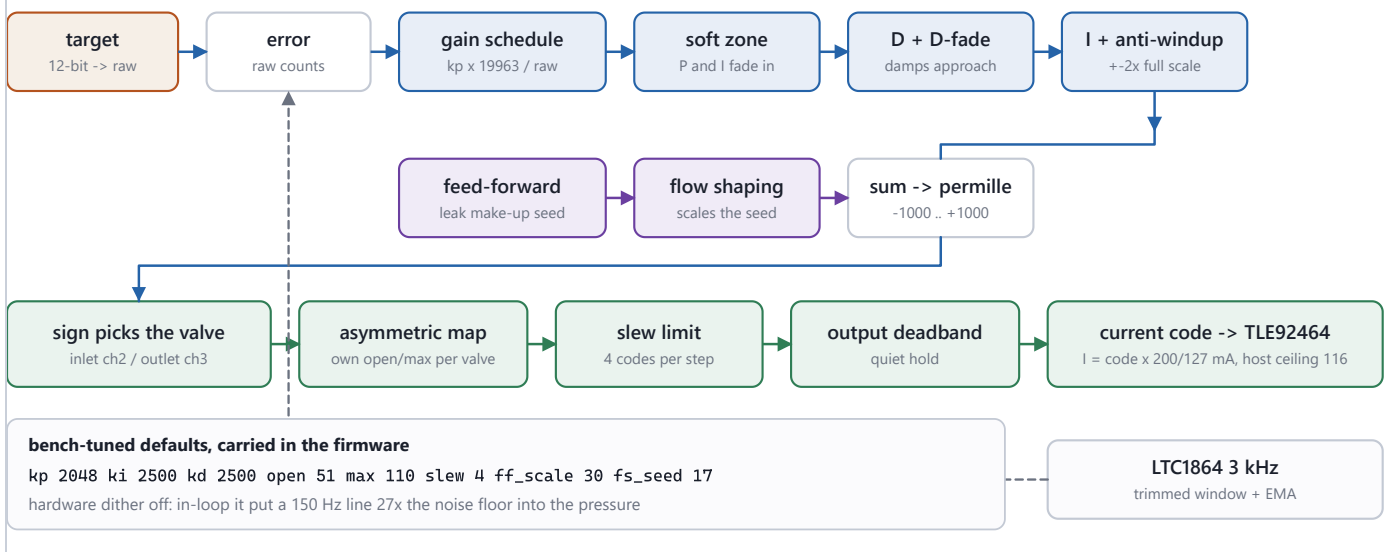


Figure 5. The loop. The output stage is asymmetric because the valves are: the inlet cracks at 80 mA, the outlet has no real authority until 87 mA, so each gets its own open and maximum code. The firmware caps every host-supplied code at 116 (182.7 mA): the coil budget is 120 (189 mA) and four codes are held back for the hardware-dither overlay that rides on top of the DC setpoint. Hardware dither defaults to zero amplitude: in a closed loop it put a 150 Hz line 27× the noise floor into the feedback for no measurable stiction benefit.

6 Firmware update

The image is broadcast once and every selected board writes it;

only start, progress poll and finish are addressed individually. Boards stay silent through the stream — eight of them acknowledging every frame would put eight frames on the bus for

each one the host sends. Progress is read back at each 896-byte block boundary, and a board that fell behind is topped up over its own unicast identifier before the next block starts.

Two guards matter more than the throughput. An update is irreversible once a board writes it, so the host refuses to broadcast unless the image's own project descriptor names the board type every selected board reported. An ID-range check is

not enough: the sixteen 7 mm boards accept a TLE image just as readily, and all sixteen would leave the bus until each was reflashed by hand. Separately, a USB flash rewrites otadata with the application, because the application offset is the factory partition while an update writes a spare slot — without that, flashing a board that has ever been updated over CAN leaves it running the old image.

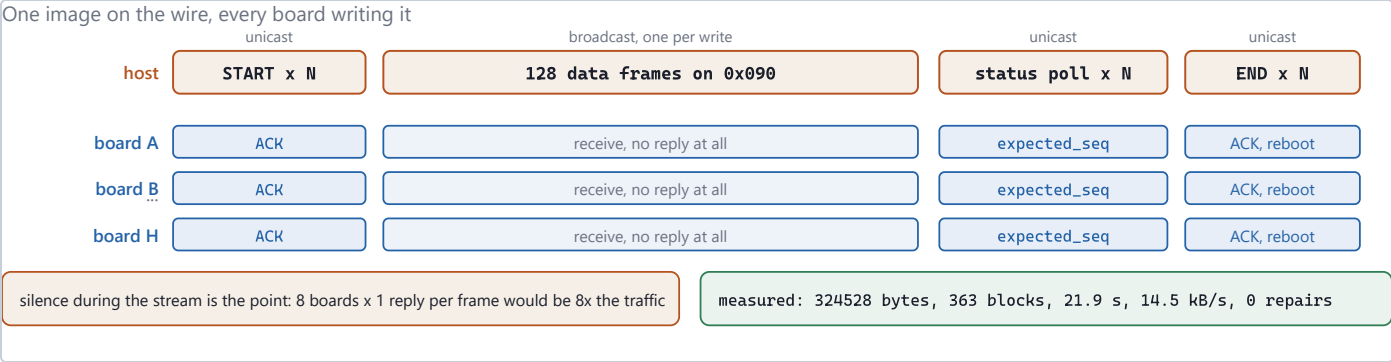


Figure 6. Broadcast update. A board's identifier lives in the NVS partition, which sits below the application in the flash layout; neither an update nor a USB flash writes anywhere near it, so neither can rename a board.

Do not batch frames into one write. The obvious optimisation — several frames per USB transaction — breaks the transfer completely. Frames written together reach the wire back to back, and broadcast image data all arrives on one identifier, which a board filters into a single receive buffer; it then has one frame time to read each out over SPI. Measured on one 896-byte

block: batches of 8 delivered 1 frame, batches of 4 delivered 2, batches of 2 delivered 3, and one frame per write delivered all 128 — at a 300 µs offered gap, faster than any batched case. The gap has to be between frames, so the pacing has to be per frame.

7 What was measured

QUANTITY	RESULT	CONDITIONS
Sync edges answered	3000 / 3000	20 s at 150 Hz, one board
Reply latency, median / p95 / max	1.07 / 1.30 / 1.91 ms	1500 cycles
Host cycle jitter, p95 / max	0.001 / 0.016 ms	headless; 0.009 / 5.0 ms with the GUI plotting
Broadcast update	324 kB in 21.9 s	14.5 kB/s, 363 blocks, 0 repairs
Receive-buffer overflows / transmit failures	0 / 0	board's own counters, whole session
Identifier across USB flash and update	unchanged	verified after both
Bench checks passing	28 / 28	tools/tle_bench.py all

8 Three defects found on the way

DEFECT	WHY IT MATTERED	FIX
MCP2515 RXnOVR never cleared	The flags latch. An overrun buffer keeps rejecting frames until software clears it, silently — the controller goes on acknowledging those frames in hardware, so the transmitter sees success while the board hears nothing.	Cleared on an unconditional 100 ms sweep. Recovery must never be gated on a condition the fault itself removes.
One-shot transmission enabled	Added as a defence against a bus-less bench. On a populated bus every board answers the sync edge at the same instant, so losing arbitration is routine, and a one-shot transmitter drops the frame rather than retrying. Only the lowest-identifier board would have reported reliably.	Off. The bus-less case is left to the link breaker, which was written around retry-forever semantics anyway.
Acknowledgement lost to a reboot	Replies are queued and drained at the end of the CAN task's pass, but a reboot runs inside the receive handler, so the task never reaches its transmit slot. An update that completed correctly on the board reported failure to the host.	Both reboot paths — finishing an update, setting an identifier — flush the queue before restarting.

9 One deliberate difference

The original firmware has no link-loss timeout: a board that stops hearing the host holds its last commanded pressure indefinitely. The new boards drop their outputs after 500 ms without a sync edge — 75 missed cycles, which no healthy host produces. Expect the eight new boards to release and the sixteen old ones to hold.

10 Reproducing this

Firmware is the repository root project, host code is under TLE_PCB/, and every figure comes from docs/make_figures.py.

- idf.py build, then idf.py -p COM8 flash
- tools/tle_bench.py all --image ... — the 28 checks above
- VEMA_TLE_flash.py, VEMA_TLE_controller.py